

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas Unitarias con Pytest				
NÚMERO DE PRÁCTICA:	03	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	27/04/2026	FECHA FIN:	01/05/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: • Una computadora personal • Lenguaje de programación Python 3.10+ • Visual Studio Code + extensión Python • Framework de pruebas Pytest (pip install pytest)					
DOCENTE(s): • Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS
OBJETIVOS: • El estudiante comprenderá la importancia de diseñar pruebas unitarias de componentes de software basados en Python. • El estudiante elaborará pruebas unitarias en Pytest para demostrar que una unidad de código cumple una especificación funcional.
TEMAS: • Pruebas Unitarias: definición y propósito • Instalación y configuración de Pytest • Estructura de una prueba unitaria: patrón AAA (Arrange–Act–Assert) • Assertions en Pytest y mensajes de error descriptivos • Pruebas parametrizadas con @pytest.mark.parametrize • Fixtures: configuración y desmontaje de contexto de prueba

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

COMPETENCIAS	C.m Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente.
---------------------	---

CONTENIDO DE LA GUÍA															
I. MARCO CONCEPTUAL															
¿Qué son las Pruebas Unitarias?															
Una prueba unitaria (unit test) es un procedimiento automatizado que verifica el comportamiento de la unidad más pequeña de código comprobable en una aplicación, generalmente una función o método, de forma aislada del resto del sistema. Las pruebas unitarias constituyen la base de la pirámide de pruebas (Cohn, 2010) y son el mecanismo más rápido y económico para detectar regresiones durante el desarrollo.															
Según Osherove (2013), una buena prueba unitaria posee tres atributos fundamentales: (1) es completamente automatizada, (2) su resultado es determinista (siempre produce el mismo resultado para los mismos datos), y (3) prueba una sola unidad lógica.															
Diferencia entre Depuración y Pruebas Unitarias															
Aspecto	<table border="1"> <thead> <tr> <th>Depuración (Debugging)</th><th>Prueba Unitaria</th></tr> </thead> <tbody> <tr> <td data-bbox="181 1173 406 1268">Propósito</td><td data-bbox="406 1173 1421 1268">Encontrar y corregir un error ya conocido</td></tr> <tr> <td data-bbox="181 1268 406 1325">Activación</td><td data-bbox="406 1268 1421 1325">Reactiva: se activa al detectar un fallo</td></tr> <tr> <td data-bbox="181 1325 406 1419">Automatización</td><td data-bbox="406 1325 1421 1419">Proactiva: se ejecuta continuamente</td></tr> <tr> <td data-bbox="181 1419 406 1514">Repetibilidad</td><td data-bbox="406 1419 1421 1514">Completamente automatizada con un framework</td></tr> <tr> <td data-bbox="181 1514 406 1570">Resultado</td><td data-bbox="406 1514 1421 1570">Repetible en milisegundos con un solo comando</td></tr> <tr> <td data-bbox="181 1570 406 1627">Resultado</td><td data-bbox="406 1570 1421 1627">Verde (PASS) o Rojo (FAIL) con mensaje claro</td></tr> </tbody> </table>	Depuración (Debugging)	Prueba Unitaria	Propósito	Encontrar y corregir un error ya conocido	Activación	Reactiva: se activa al detectar un fallo	Automatización	Proactiva: se ejecuta continuamente	Repetibilidad	Completamente automatizada con un framework	Resultado	Repetible en milisegundos con un solo comando	Resultado	Verde (PASS) o Rojo (FAIL) con mensaje claro
Depuración (Debugging)	Prueba Unitaria														
Propósito	Encontrar y corregir un error ya conocido														
Activación	Reactiva: se activa al detectar un fallo														
Automatización	Proactiva: se ejecuta continuamente														
Repetibilidad	Completamente automatizada con un framework														
Resultado	Repetible en milisegundos con un solo comando														
Resultado	Verde (PASS) o Rojo (FAIL) con mensaje claro														
Propósito	Verificar que el código cumple la especificación														
Activación	Proactiva: se ejecuta continuamente														
Automatización	Completamente automatizada con un framework														
Repetibilidad	Repetible en milisegundos con un solo comando														
Resultado	Verde (PASS) o Rojo (FAIL) con mensaje claro														

¿Qué es Pytest?

Pytest es el framework de pruebas más popular del ecosistema Python (Okken, 2022). A diferencia del módulo estándar unittest, Pytest no requiere que las pruebas hereden de una clase base; basta con escribir funciones cuyo nombre comience con test_. Pytest detecta automáticamente estos archivos y funciones mediante su sistema de descubrimiento (discovery).

Sus ventajas principales son:

- **Sintaxis minimalista:** se usan assertions nativas de Python (assert) en lugar de métodos especiales.
- **Mensajes de error detallados:** al fallar un assert, Pytest muestra el valor real vs. el esperado.
- **Pruebas parametrizadas:** el decorador @pytest.mark.parametrize permite ejecutar la misma lógica con múltiples conjuntos de datos.

- **Fixtures:** mecanismo para preparar y limpiar contexto de prueba de forma reutilizable.
- **Plugins:** ecosistema extensible con más de 1 000 plugins (pytest-cov, pytest-html, etc.).

Documentación oficial de Pytest: <https://docs.pytest.org/en/stable/>

Instalación:

```
pip install pytest
pytest --version # Verificar instalación
```

El Patrón AAA: Arrange – Act – Assert

El patrón AAA (Beck, 2002) es la estructura canónica de cualquier prueba unitaria:

Fase	Descripción
Arrange	Prepara los datos de entrada, objetos y condiciones previas necesarias para la prueba.
Act	Ejecuta la unidad bajo prueba (función o método) con los datos preparados.
Assert	Verifica que el resultado obtenido coincide con el resultado esperado.

Ejemplo mínimo ilustrativo:

```
# test_suma.py
def sumar(a, b):
    return a + b

def test_sumar_enteros_positivos():
    # Arrange
    a, b = 3, 5
    # Act
    resultado = sumar(a, b)
    # Assert
    assert resultado == 8
```

Para ejecutar las pruebas desde la terminal:

```
pytest test_suma.py -v

# Salida esperada:
# test_suma.py::test_sumar_enteros_positivos PASSED [ 100%]
# 1 passed in 0.02s
```

Pruebas Parametrizadas con @pytest.mark.parametrize

Cuando una función debe probarse con múltiples combinaciones de entrada/salida, el decorador @pytest.mark.parametrize evita la duplicación de código. Esto es especialmente relevante cuando se aplican técnicas de diseño de casos de prueba como la Partición de Equivalencia o el Análisis de Valores Límite (Spillner & Linz, 2021).

```
import pytest
# test_suma.py
def sumar(a, b):
    return a + b

@pytest.mark.parametrize("a, b, esperado", [
    (2, 3, 5),      # caso normal
    (0, 0, 0),      # caso borde: ceros
    (-1, 1, 0),     # caso borde: negativos
    (100, 200, 300), # caso grande
])

def test_sumar_parametrizado(a, b, esperado):
    assert sumar(a, b) == esperado
```

Fixtures en Pytest

Un fixture es una función decorada con `@pytest.fixture` que proporciona un contexto de prueba reutilizable. Se utiliza cuando varias pruebas comparten la misma configuración inicial (por ejemplo: crear un objeto, conectar a una base de datos, preparar datos de ejemplo). Pytest inyecta el fixture automáticamente al declararlo como parámetro de la función de prueba (Okken, 2022).

```
import pytest

@pytest.fixture
def cuenta_bancaria():
    """Fixture: retorna una cuenta con saldo inicial de 1000."""
    return {"saldo": 1000.0, "titular": "Ana Flores"}

def test_saldo_inicial(cuenta_bancaria):
    assert cuenta_bancaria["saldo"] == 1000.0

def test_deposito(cuenta_bancaria):
    cuenta_bancaria["saldo"] += 500
    assert cuenta_bancaria["saldo"] == 1500.0
```

Convenciones de nomenclatura en Pytest

Para que Pytest descubra automáticamente los archivos y funciones de prueba, se deben seguir estas convenciones:

- **Archivos de prueba:** deben nombrarse `test_*.py` o `*_test.py`
- **Funciones de prueba:** deben comenzar con `test_`
- **Clases de prueba (opcional):** deben comenzar con `Test` (sin método `__init__`)

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejercicio 3.1 – Primera prueba unitaria con Pytest (nivel básico)

En este ejercicio crearemos una función simple de conversión de temperatura y escribiremos pruebas unitarias completas aplicando el patrón AAA.

Paso 1: Crear el proyecto

```
# Estructura del proyecto /LAB_03/  
LAB_03/  
├── conversor.py      # módulo bajo prueba  
└── test_conversor.py # archivo de pruebas
```

Paso 2: Escribir el módulo conversor.py

```
# conversor.py  
# Módulo de conversión de temperaturas  
  
def celsius_a_fahrenheit(celsius):  
    """Convierte grados Celsius a Fahrenheit.  
    Fórmula: F = (C * 9/5) + 32  
    """  
    return (celsius * 9 / 5) + 32  
  
def fahrenheit_a_celsius(fahrenheit):  
    """Convierte grados Fahrenheit a Celsius.  
    Fórmula: C = (F - 32) * 5/9  
    """  
    return (fahrenheit - 32) * 5 / 9
```

Paso 3: Escribir el archivo test_conversor.py con el patrón AAA

```
# test_conversor.py  
import pytest  
from conversor import celsius_a_fahrenheit, fahrenheit_a_celsius  
  
# — Pruebas para celsius_a_fahrenheit —  
  
def test_celsius_a_fahrenheit_punto_congelacion():  
    # Arrange  
    celsius = 0  
    # Act  
    resultado = celsius_a_fahrenheit(celsius)  
    # Assert  
    assert resultado == 32.0, f"Esperado 32.0 pero obtuvo {resultado}"  
  
def test_celsius_a_fahrenheit_punto_ebullicion():  
    # Arrange  
    celsius = 100  
    # Act  
    resultado = celsius_a_fahrenheit(celsius)  
    # Assert  
    assert resultado == 212.0  
  
def test_celsius_a_fahrenheit_temperatura_negativa():  
    assert celsius_a_fahrenheit(-40) == -40.0 # punto de equivalencia
```

```
# — Pruebas parametrizadas —

@pytest.mark.parametrize("celsius, fahrenheit", [
    (0, 32.0),
    (100, 212.0),
    (-40, -40.0),
    (37, 98.6), # temperatura corporal normal
    (-273.15, -459.67), # cero absoluto
])
def test_celsius_a_fahrenheit_parametrizado(celsius, fahrenheit):
    resultado = celsius_a_fahrenheit(celsius)
    assert abs(resultado - fahrenheit) < 0.01, (
        f"celsius={celsius}: esperado {fahrenheit}, obtuvo {resultado}"
    )
```

Paso 4: Ejecutar las pruebas desde la terminal

```
cd LAB_03
pytest test_convertor.py -v

# Salida esperada:
# test_convertor.py::test_celsius_a_fahrenheit_punto_congelacion PASSED
# test_convertor.py::test_celsius_a_fahrenheit_punto_ebullicion PASSED
# test_convertor.py::test_celsius_a_fahrenheit_temperatura_negativa PASSED
# test_convertor.py::test_celsius_a_fahrenheit_parametrizado[0-32.0] PASSED
# test_convertor.py::test_celsius_a_fahrenheit_parametrizado[100-212.0] PASSED
# ...
# 8 passed in 0.05s
```

Tabla de casos de prueba del Ejercicio 3.1:

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Punto de congelación del agua	celsius = 0	32.0 °F	PASS
TC-02	Punto de ebullición del agua	celsius = 100	212.0 °F	PASS
TC-03	Punto de equivalencia (-40)	celsius = -40	-40.0 °F	PASS
TC-04	Temperatura corporal normal	celsius = 37	98.6 °F	PASS
TC-05	Cero absoluto (Kelvin 0 K)	celsius = -273.15	-459.67 °F	PASS

Ejercicio 3.2 – Pruebas unitarias con Fixtures y pruebas de excepciones (nivel intermedio)

En este ejercicio aplicaremos fixtures para probar una clase BankAccount y aprenderemos a verificar que el programa lanza excepciones cuando recibe entradas inválidas, usando pytest.raises.

Archivo bank_account.py:

```
# bank_account.py

class SaldoInsuficienteError(Exception):
    """Se lanza cuando se intenta retirar más del saldo disponible."""
    pass

class MontoInvalidoError(Exception):
    """Se lanza cuando el monto es cero o negativo."""
    pass

class BankAccount:
    def __init__(self, titular: str, saldo_inicial: float = 0.0):
        if saldo_inicial < 0:
            raise MontoInvalidoError("El saldo inicial no puede ser negativo.")
        self.titular = titular
        self._saldo = saldo_inicial

    @property
    def saldo(self) -> float:
        return self._saldo

    def depositar(self, monto: float) -> None:
        if monto <= 0:
            raise MontoInvalidoError(f"Monto inválido: {monto}. Debe ser positivo.")
        self._saldo += monto

    def retirar(self, monto: float) -> None:
        if monto <= 0:
            raise MontoInvalidoError(f"Monto inválido: {monto}. Debe ser positivo.")
        if monto > self._saldo:
            raise SaldoInsuficienteError(
                f"Saldo insuficiente: tiene S/.{self._saldo}, intenta retirar S/.{monto}"
            )
        self._saldo -= monto
```

Archivo test_bank_account.py:

```
# test_bank_account.py
import pytest

from bank_account import BankAccount, SaldoInsuficienteError, MontoInvalidoError

# — Fixture —————
@pytest.fixture
```

```
def cuenta():
    """Fixture: retorna una BankAccount con S/.1000 de saldo inicial."""
    return BankAccount(titular="Luis Quispe", saldo_inicial=1000.0)

# --- Pruebas: saldo y depósito ---
def test_saldo_inicial(cuenta):
    assert cuenta.saldo == 1000.0

def test_deposito_incrementa_saldo(cuenta):
    cuenta.depositar(500.0)
    assert cuenta.saldo == 1500.0

def test_retiro_decrementa_saldo(cuenta):
    cuenta.retirar(200.0)
    assert cuenta.saldo == 800.0

# --- Pruebas: excepciones ---
def test_retiro_excede_saldo_lanza_excepcion(cuenta):
    with pytest.raises(SaldoInsuficienteError):
        cuenta.retirar(2000.0)

def test_deposito_monto_negativo_lanza_excepcion(cuenta):
    with pytest.raises(MontoInvalidoError):
        cuenta.depositar(-100.0)

def test_retiro_monto_cero_lanza_excepcion(cuenta):
    with pytest.raises(MontoInvalidoError):
        cuenta.retirar(0)

def test_cuenta_saldo_inicial_negativo_lanza_excepcion():
    with pytest.raises(MontoInvalidoError):
        BankAccount("Error", saldo_inicial=-500)

# --- Prueba parametrizada: múltiples retiros ---
@pytest.mark.parametrize("retiro, saldo_esperado", [
    (100.0, 900.0),
    (500.0, 500.0),
    (999.99, 0.01),
    (1000.0, 0.0),
])
def test_retiros_validos(cuenta, retiro, saldo_esperado):
    cuenta.retirar(retiro)
    assert abs(cuenta.saldo - saldo_esperado) < 0.001
```

Tabla de casos de prueba del Ejercicio 3.2:

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Saldo inicial correcto	saldo_inicial=1000	cuenta.saldo == 1000.0	PASS

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 9

TC-02	Depósito incrementa saldo	depositar(500)	saldo == 1500.0	PASS
TC-03	Retiro decrementa saldo	retirar(200)	saldo == 800.0	PASS
TC-04	Retiro mayor al saldo lanza excepción	retirar(2000)	SaldoInsuficienteError	PASS
TC-05	Depósito negativo lanza excepción	depositar(-100)	MontolInvalidoError	PASS
TC-06	Retiro de monto cero lanza excepción	retirar(0)	MontolInvalidoError	PASS
TC-07	Saldo inicial negativo lanza excepción	saldo_inicial=-500	MontolInvalidoError	PASS
TC-08	Retiro parametrizado: S/.100	retirar(100)	saldo == 900.0	PASS
TC-09	Retiro parametrizado: S/.1000 (saldo exacto)	retirar(1000)	saldo == 0.0	PASS

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Para cada uno de los siguientes ejercicios:

- Cree el programa en Python (módulo bajo prueba)
- Diseñe los casos de prueba necesarios en una tabla (Id, Descripción, Entrada, Resultado Esperado)
- Implemente las pruebas unitarias con Pytest aplicando el patrón AAA
- Valide que su programa pasa todos los casos de prueba (captura de pantalla del resultado pytest -v)

1. Calculadora Básica

Enunciado: Escriba un módulo calculadora.py que implemente las cuatro operaciones aritméticas básicas: suma, resta, multiplicación y división. Para la división, el programa debe manejar adecuadamente la división por cero lanzando una excepción de tipo ValueError con el mensaje “No se puede dividir entre cero”.

Objetivo de prueba: Verificar que cada operación produce el resultado correcto para entradas enteras, decimales y negativas; y que la división por cero es manejada como excepción.

El informe debe contener el código fuente:

- calculadora.py
- test_calculadora.py

Guía de casos de prueba mínimos requeridos:

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Suma de dos enteros positivos	(5, 3)	8	
TC-02	Suma con número negativo	(-4, 4)	0	
TC-03	Resta resultado positivo	(10, 3)	7	
TC-04	Resta resultado negativo	(3, 10)	-7	
TC-05	Multiplicación por cero	(5, 0)	0	
TC-06	Multiplicación de decimales	(2.5, 4)	10.0	
TC-07	División exacta	(10, 2)	5.0	
TC-08	División por cero lanza excepción	(5, 0)	ValueError	
TC-09	División decimal	(7, 2)	3.5	

2. Validador de Contraseñas

Enunciado: Implemente un módulo validador.py con la función validar_contrasena(contrasena: str) -> dict que evalúe la fortaleza de una contraseña según las siguientes reglas:

- Longitud mínima de 8 caracteres.
- Contiene al menos una letra mayúscula.
- Contiene al menos una letra minúscula.

- Contiene al menos un dígito numérico.
- Contiene al menos un carácter especial de entre: ! @ # \$ % ^ & *

La función debe retornar un diccionario con la clave "valida" (True/False) y la clave "errores" (lista de strings con los criterios no cumplidos). Si la contraseña cumple todos los criterios, "errores" debe ser una lista vacía.

Objetivo de prueba: Comprobar que el validador detecta correctamente cada regla incumplida de forma individual y en combinación; y que acepta contraseñas que cumplen todos los criterios.

El informe debe contener el código fuente:

- validador.py
- test_validador.py

Guía de casos de prueba mínimos requeridos:

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Contraseña completamente válida	"Segura#1"	{valida: True, errores: []}	
TC-02	Contraseña muy corta (<8 caracteres)	"Ab1!"	valida: False, error longitud	
TC-03	Sin letra mayúscula	"segura#1"	valida: False, error mayúscula	
TC-04	Sin letra minúscula	"SEGURA#1"	valida: False, error minúscula	
TC-05	Sin dígito numérico	"Segura##"	valida: False, error dígito	
TC-06	Sin carácter especial	"Segura12"	valida: False, error especial	
TC-07	Contraseña vacía (múltiples errores)	""	valida: False, 5 errores	
TC-08	Exactamente 8 caracteres válidos	"aB1!cDe2"	{valida: True, errores: []}	

TC-09	Parametrizado: múltiples contraseñas débiles	ver código	valida: False en todos	
-------	---	------------	------------------------	--

3. Simulador de Cajero Automático con Pruebas Unitarias

Enunciado: Retome el ejercicio del Mini Cajero Automático de la Guía LAB_01. Esta vez, refactorice la lógica de negocio en una clase Atm (archivo atm.py) con los métodos consultar_saldo(), depositar(monto) y retirar(monto). Aplique el mismo esquema de excepciones de la clase BankAccount del Ejercicio 3.2 de esta guía. Luego, escriba un archivo test_atm.py que valide toda la lógica usando Pytest, fixtures y parametrize.

Objetivo de prueba: Verificar que el flujo completo del cajero es correcto: consulta de saldo, depósitos válidos e inválidos, retiros válidos e inválidos (incluyendo casos límite como retirar exactamente el saldo disponible), y el manejo correcto de todas las excepciones.

El informe debe contener el código fuente:

- atm.py (clase Atm con lógica de negocio, sin llamadas a input())
- test_atm.py (suite completa de pruebas Pytest con fixture y parametrize)

Guía de casos de prueba mínimos requeridos (diseñe al menos 12 casos):

Id.	Descripción	Entrada	Resultado Esperado	Resultado Real
TC-01	Saldo inicial correcto	saldo_inicial=1000	saldo == 1000.0	
TC-02	Depósito válido	depositar(500)	saldo == 1500.0	
TC-03	Retiro válido	retirar(300)	saldo == 700.0	
TC-04	Retiro exacto al saldo disponible	retirar(1000)	saldo == 0.0	
TC-05	Retiro mayor al saldo	retirar(1001)	SaldoInsuficienteError	
TC-06	Depósito de monto negativo	depositar(-200)	MontoInvalidoError	
TC-07	Depósito de monto cero	depositar(0)	MontoInvalidoError	
TC-08	Retiro de monto negativo	retirar(-50)	MontoInvalidoError	

TC-09	Saldo inicial negativo	saldo_inicial=-500	MontolInvalidoError	
TC-10	Múltiples depósitos acumulados (parametrized)	[100, 200, 300]	saldo == 1600.0	
TC-11	Múltiples retiros acumulados (parametrized)	[100, 200, 300]	saldo == 400.0	
TC-12	Consulta de saldo no modifica el estado	consultar_saldo() x3	saldo inalterado	

I. CUESTIONARIO

1. ¿Cuál es la diferencia fundamental entre una prueba unitaria y una prueba de integración? Proporcione un ejemplo concreto para cada tipo en el contexto de un sistema bancario.
2. Explique el principio F.I.R.S.T. de las pruebas unitarias (Fast, Independent, Repeatable, Self-validating, Timely). ¿Por qué el atributo "Independent" es especialmente importante al usar fixtures en Pytest?
3. En el patrón AAA (Arrange–Act–Assert), ¿qué problemas puede ocasionar colocar múltiples assertions (assert) en una misma función de prueba? ¿Cuándo se considera aceptable hacerlo?
4. Con respecto a Test-Driven Development (TDD) y sus tres fases (Red–Green–Refactor). ¿De qué manera las pruebas unitarias con Pytest facilitan la adopción de TDD? Incluya un ejemplo sencillo en Python.
5. ¿Qué es la cobertura de código (code coverage) y cómo se mide con el plugin pytest-cov? Investigue las métricas statement coverage y branch coverage. ¿Una cobertura del 100% garantiza que el software no tiene errores? Justifique su respuesta.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- [1] Beck, K. (2002). Test Driven Development: By Example. Addison-Wesley Professional.
- [2] Cohn, M. (2010). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley Professional.
- [3] Okken, B. (2022). Python Testing with pytest: Simple, Rapid, Effective, and Scalable (2nd ed.). Pragmatic Bookshelf.
- [4] Osherove, R. (2013). The Art of Unit Testing (2nd ed.). Manning Publications.
- [5] Spillner, A., & Linz, T. (2021). Software Testing Foundations: A Study Guide for the Certified Tester Exam (5th ed.). Rocky Nook.
- [6] Myers, G. J., Sandler, C., & Badgett, T. (2012). The Art of Software Testing (3rd ed.). John Wiley & Sons.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 14

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y proporcionan explicaciones claras, incluyen citas.	La mayoría de las respuestas son correctas, pero algunas carecen de la profundidad esperada, pocas citas.	Algunas respuestas son correctas, pero incompletas o con falta de claridad en las explicaciones, citas no relevantes.	La mayoría de las respuestas son escuetas y/o incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	No presenta el cuestionario.



Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.
--	---	---	--	---	-------------------------